The Lost Eject: Claude and Uppaal Debug a Box-Sorting Controller

Author Names

Institution

Abstract. A modeller loaded an existing Uppaal model of a conveyor belt that sorts red and black boxes and asked the AI assistant Claude, connected to the Uppaal editor through a tool interface, whether the controller removes every black box and lets every red box pass. Claude formalised the requirement, Uppaal found that it fails, and located the cause with a handful of diagnostic reachability queries. Claude provide the explanation: the controller has only one timer, so a second black box overwrites the pending eject of the first. Claude replaced the timer by one timer per box in transit and verified the fix for up to eight black boxes, after introducing scalar sets so that Uppaal's symmetry reduction made the larger instances tractable. The whole case took about eight minutes of assistant time. We summarise the session and reflect on diagnosis by queries, checking that properties can fail, modelling for scalability, and the robustness of the tool interface.

1 Setting

The model is in the style of the LEGO® MINDSTORMS sorter case study [3]. Boxes are timed automata `RedBox` and `BlackBox` whose clock `pos` is the position on the belt (Fig. 1). A `Guard` places boxes at least 10 time units apart. Between positions 9 and 18 the colour is sensed over the urgent channels `red` and `blck`; between 81 and 90 a box can be pushed off by the `Piston`, ending in `off`; a box that reaches 90 ends in `end`. The control software consists of two tasks: `TaskMAIN` receives the colour, and on `blck` resets a clock `x` and sets `active`; `TaskPUSH` then waits until `x==ctime`, with `ctime = 75`, and sends `eject` to the piston. The modeller's question was: *"The intended behavior is that all black boxes should be removed – ending in off – and all red boxes should make it to end. Is this correct and can you check this?"* Through the tool interface [1] Claude could read and edit the model, add queries and run the verifier; it never saw the GUI.

2 Finding the Bug with Diagnostic Queries

Claude formalised the requirement as two safety properties and added a reachability check that the intended outcome is possible at all (Table 1, top). With two boxes of each colour, Uppaal reported that a black box can reach `end`, while red

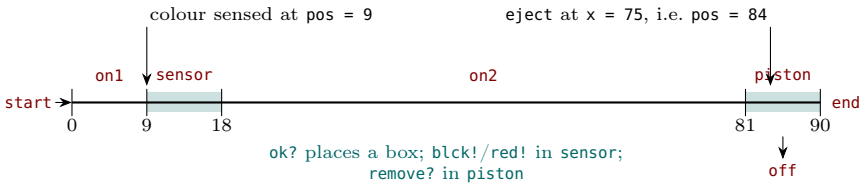


Fig. 1. The belt as seen by one box. The clock `pos` measures the position; shaded zones are where a box can be sensed and removed. Since the sensing channels are urgent and always enabled, a box is sensed as soon as it reaches position 9.

Table 1. Queries proposed by Claude on the original model (two red, two black boxes) and UPPAAL's verdicts.

Query	Meaning	Result
`A[] not (B1.end || B2.end)`	no black box reaches the end	violated
`A[] not (R1.off || R2.off)`	no red box is removed	holds
`E<> B1.off && B2.off && R1.end && R2.end`	intended outcome possible	holds
`E<> B1.end && B2.start`	B1 escapes on its own	no
`E<> B1.end && B2.off`	B1 escapes, B2 is ejected	yes
`E<> TaskPUSH.wait && B1.on2 && B2.on2`	two in transit, one timer	yes

boxes are never removed. Rather than inspect a counterexample trace, Claude posed three diagnostic queries as experiments (Table 1, bottom): a black box never escapes while the other black box is still waiting to be placed, but it can escape while the other one is ejected, and two black boxes can be between sensor and piston while only one eject is pending.

From these answers and the timing constants Claude reconstructed the scenario of Fig. 2. The travel time from sensor to piston (75) is much longer than the minimal spacing of boxes (10), so a second black box can be sensed while the first is still in transit. `TaskMAIN` then resets the single clock `x`, which postpones the only pending eject. The first box passes position 90 before the eject happens, and the delayed eject removes the second box. It is not a race at the piston: the controller forgets the earlier box. The same reasoning shows why red boxes are safe: an eject always happens 75 after some black box was sensed, when that box is at position 84, and the spacing of 10 keeps every other box out of the piston zone.

3 The Fix and its Verification

The controller must remember an eject time for every black box in transit. Claude replaced the clock `x` and flag `active` by arrays, gave `TaskPUSH` a parameter so that UPPAAL instantiates one copy per timer, and let `TaskMAIN` start a free timer on each detection:



Fig. 2. How a black box escapes, for a spacing $10 \le d < 75$ (drawn with $d = 20$). The second detection resets the only timer, so the eject that was due at $t + 75$ happens at $t + d + 75$, after B1 has left the piston zone.

```
const int NT = 8;
typedef scalar[NT] tid_t;
bool  active[tid_t];
clock x[tid_t];
```

The `blck?` edge of `TaskMAIN` now has the select `i : tid_t`, the guard `!active[i]` and the update `x[i] = 0, active[i] = true`. The template `TaskPUSH(const tid_t id)` waits in `wait` under the invariant `x[id] <= ctime` and leaves with `eject!` when `x[id] == ctime`, setting `active[id] = false`. The requirement was checked with three properties, now quantified over the boxes:

```
A[] forall (i : bid_t) not BlackBox(i).end
A[] forall (j : rid_t) not RedBox(j).off
A[] deadlock imply ((forall (i : bid_t) BlackBox(i).off) &&
                    (forall (j : rid_t) RedBox(j).end))
```

The third property captures the full requirement as a safety property. Since the guard may wait forever before placing a box, "eventually every box is sorted" does not hold; but a box still in `start` can always be placed, so the system can only come to rest after all boxes have been placed and have left the belt. Two reachability queries complement them: the intended final state is reachable, and all black boxes can be in transit at the same time, so the case with every timer busy is actually exercised.

The modeller first asked for three boxes of each colour; all checks passed. Claude then reduced the number of timers to one, which behaves like the original controller, and confirmed that the three safety properties fail: the properties do detect the bug. The modeller then asked for `NT = 8` and more black boxes. Table 2 shows the results.

Scaling needed a model change. With named instances of five black and three red boxes, the first run in the editor did not finish within the interface's time-out. Claude attributed this to the interleavings of identical boxes and timers and rewrote the model with scalar sets [2]: boxes and timers became parameterised templates over `scalar` types, instantiated automatically, and the queries were rewritten with `forall`. One part of the fix had to change for this. The first version picked the lowest-numbered free timer with a function `firstFree()`; that ordering breaks the symmetry, so Claude replaced it by a nondeterministic

Table 2. Verification of the fixed controller with `verifyta` 5.1 (symmetry reduction, breadth-first). States: largest number stored for any of the six queries; time: all six queries, including start-up.

timers	black	red	3 safety properties	3 reachability checks	states	time
8	5	3	hold	hold	20 160	3 s
8	6	3	hold	hold	34 040	5 s
8	8	3	hold	hold	69 745	14 s
4	5	3	**violated**	hold	37 923	3 s

`select` over free timers. With symmetry reduction, five black boxes need about $2 \cdot 10^4$ states and three seconds.

When the interface blocked. After the time-out the verifier kept running and the tool interface answered every request with "busy", even after the modeller stopped the verification in the GUI. Claude wrote the symmetric model to a file, found the command-line verifier `verifyta` of the same UPPAAL version on the machine, and produced Table 2 with it. When the editor was available again, Claude loaded the model and re-ran the six queries there for five black and three red boxes, with the same verdicts.

4 Reflections

Queries as experiments. Claude did not read a counterexample trace. It localised the fault by asking UPPAAL small reachability questions whose answers separate hypotheses: one black box alone, two black boxes with one ejected, two in transit with one timer. Each answer is a verified fact about the model, so the diagnosis rests on UPPAAL's verdicts and the timing constants, not only on Claude's reading of the model. The explanation of Fig. 2 is Claude's; it was not confirmed against a concrete trace, which the interface now offers.

Checking that properties can fail. A property that holds may hold for the wrong reason. Re-running the properties on deliberately weakened controllers (one timer, four timers for five boxes) showed that they detect the bug. This mutation step is cheap with an assistant that edits the model and is worth making routine. Its by-product was an unexplained observation: with too few timers a red box can also be removed, which Claude reported without tracing it.

Requirements need care to formalise. The modeller's requirement mixes safety (no black box at the end, no red box removed) with progress (every box is eventually sorted). The progress part fails for a trivial reason, an idle guard, and Claude expressed it as a safety property over the resting states instead. Such choices change what is proved; they should be visible to the modeller, as they were here.

Bounded results and an informal bound. The results hold for the box counts in Table 2. That eight timers suffice for any number of boxes rests on an argument:

a black box occupies a timer for 75 time units and boxes are at least 10 apart, so at most eight are in transit. The argument is consistent with the results, including the reachable state with eight boxes in transit, but it was not proved; a parameterised or inductive argument in UPPAAL would close the gap.

Scalability is a modelling task. Symmetry reduction is built into UPPAAL, but using it required rewriting the model and removing an ordering that the first fix had introduced. The step from "the check times out" to "use scalar sets, and drop `firstFree()`" is the kind of expert knowledge an assistant can supply.

The interface must survive long checks. A verification that runs into the interface's time-out should be cancelled with it; here it blocked every further request. The fallback to `verifyta` worked because the same UPPAAL version was installed, and the provenance of each number (editor or command line, which model file) had to be stated explicitly.

Division of labour. The modeller asked the question, chose the scale and stopped the stuck verification; UPPAAL provided every verdict and state count; Claude formalised, diagnosed, fixed and re-modelled. The analysis of the original model took under a minute, the fix with its checks one and a half minutes, and scaling about six minutes, most of it spent on the blocked interface. This summary was written by Claude from the session record; all verdicts and state counts in it are UPPAAL's.

References

1. Behrmann, G., David, A., Larsen, K.G.: A tutorial on UPPAAL. In: SFM-RT 2004. LNCS, vol. 3185, pp. 200–236. Springer (2004)
2. Hendriks, M., Behrmann, G., Larsen, K.G., Niebert, P., Vaandrager, F.: Adding symmetry reduction to UPPAAL. In: FORMATS 2003. LNCS, vol. 2791, pp. 46–59. Springer (2004)
3. Iversen, T.K., Kristoffersen, K.J., Larsen, K.G., Laursen, M., Madsen, R.G., Mortensen, S.K., Pettersson, P., Thomasen, C.B.: Model-checking real-time control programs: Verifying LEGO MINDSTORMS systems using UPPAAL. In: ECRTS 2000, pp. 147–155. IEEE (2000)